



Securing the Agentic Stack: Docker Hardened Images and Supply Chain Security



Whale, Hello There 🙋



Ajeet Raina
Developer Advocate

@ajeetsraina



Agenda

01

Why Supply Chain Security Matters Now

02

Building blocks

03

Standard Image vs DHI

04

Securing your CI Pipeline

05

Securing the Agentic Stack

06

Wrap Up

Workshop – labs

Lab 1: Prerequisite & Setup – (2 min)

- Accessing the Simspace

Lab 2: Secure Software Supply Chain – (15–20 min)

- Why supply chain security matters now
- Agents expands your attack surface
- Building Blocks – SBOM, VEX and SLSA
- Standard Image vs Docker Hardened Images (DHI)

Lab 3: Securing your CI Pipeline – (45 min)

- Docker Scout Build policies
- CI policy enforcement

Lab 4: Securing the Agentic Stack – (30 min)

- Overview of Agentic Supply Chain
- MCP servers on DHI
- Your Security Framework



Why Supply Chain Security Matters?

Especially when agents are doing the pulling.

Agents expand your attack surface, automatically

TRADITIONAL WORKFLOW

- 👤 Developer pulls base image - *manually, with intent*
- 👤 Developer installs dependencies - *reviewed in a PR*
- 👤 CI pipeline runs - *with human-authored config*

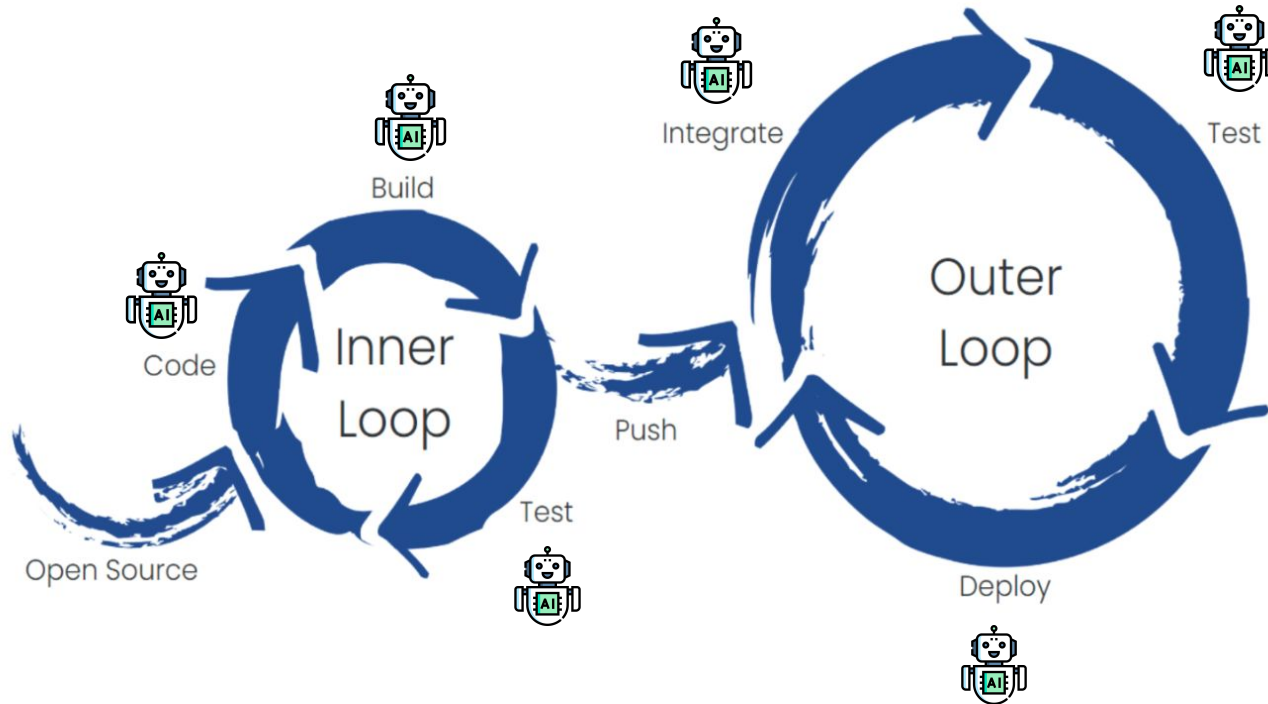
AGENTIC WORKFLOW

- 📦 Agent pulls base image - **autonomously**
- 📦 Agent installs packages - **no human review**
- ⚠️ Agent invokes external tools - **with real credentials**
- ⚠️ Agent modifies Dockerfile - **mid-pipeline**

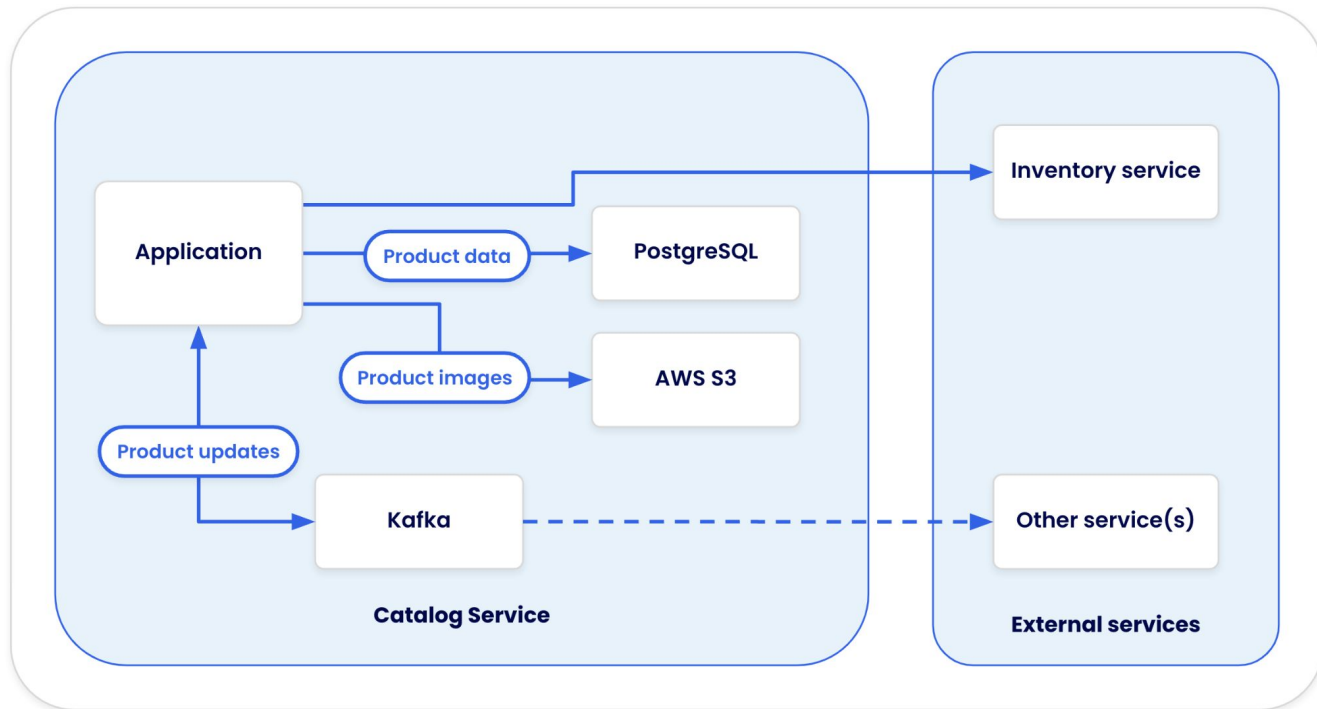
| "The better the agent, the bigger the blast radius."

The Agentic Workflow

Agent now runs inside both loops, with your permissions. The attack surface is no longer just what you depends on.



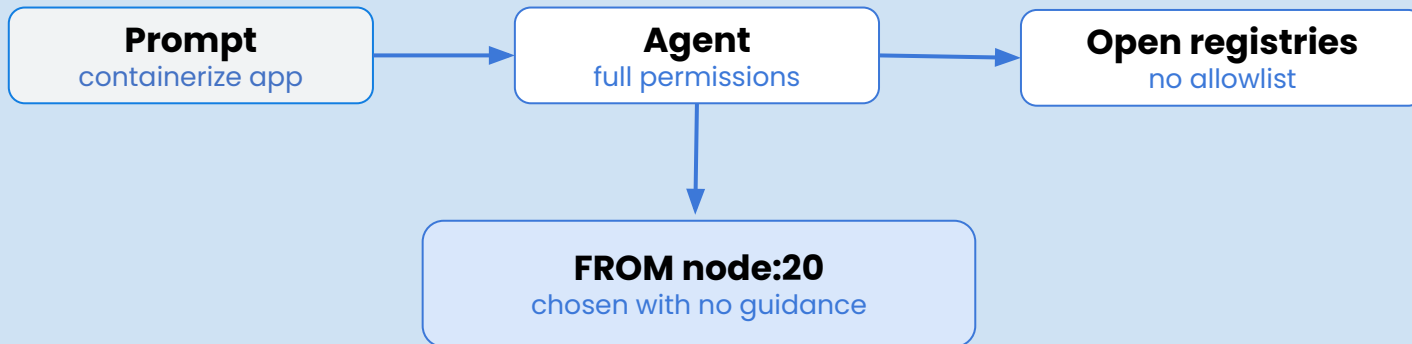
A Product Catalog Sample Demo



Agent without a Sandbox

Your host

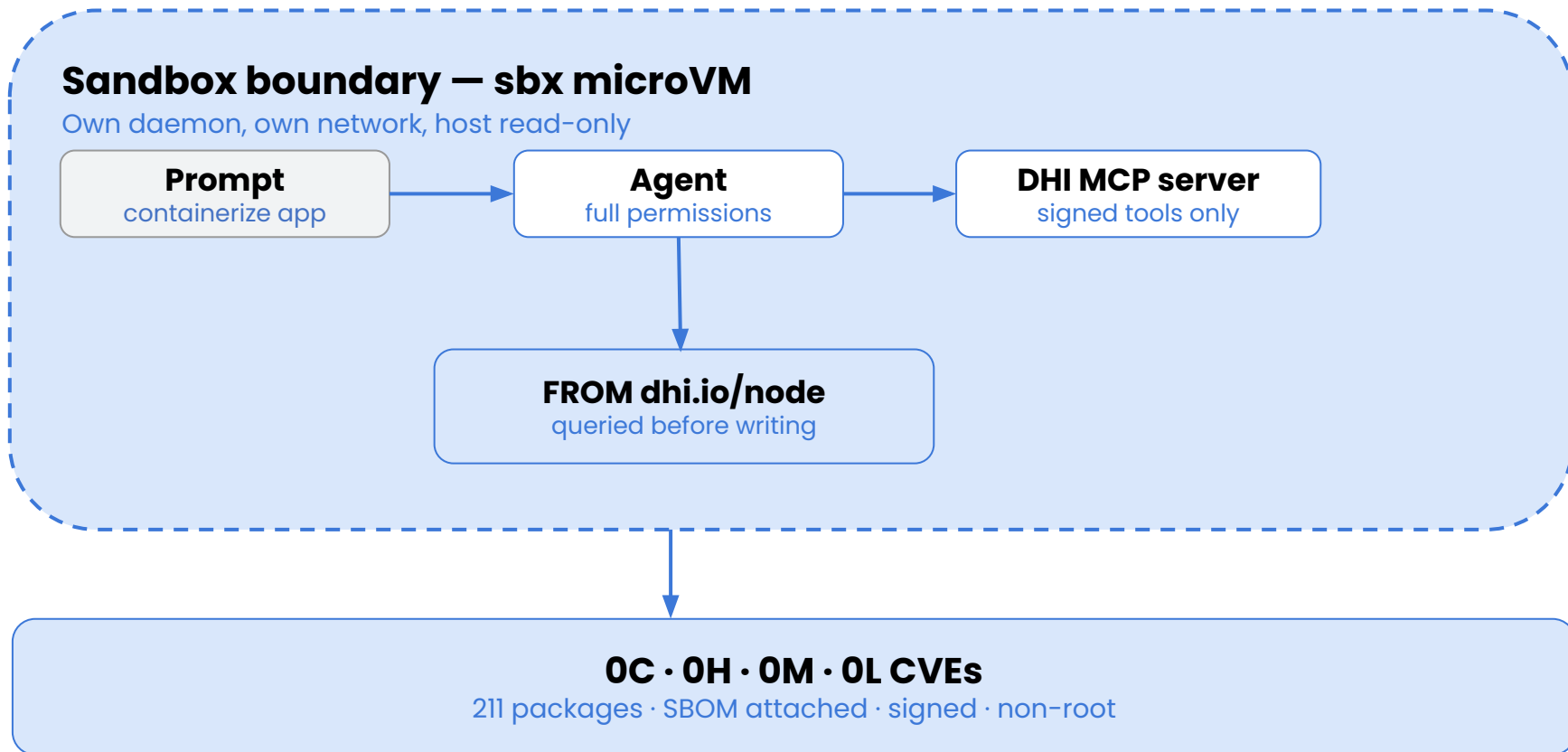
Host daemon, host credentials, no boundary



0C · 6H · 30M · 54L CVEs

431 packages · no SBOM · no attestation · root user

Agent with a Sandbox



Catalog service – where vulnerabilities enter

WITHOUT SBX – AGENT PICKS FREELY

CATALOG SERVICE

Application
FROM node:20

2C · 26H
806
pkgs

PostgreSQL
postgres:16

8H+

AWS S3
aws-sdk v3

CVEs

Kafka
confluentinc/cp-kafka:7.6

12H+

Inventory service

Other services

Total stack exposure

2C · 46H+ CVEs

WITH SBX + DHI MCP – AGENT QUERIES FIRST

CATALOG SERVICE

Application
FROM dhi.io/node:24-debian13

0C · 0H
211 pkgs

PostgreSQL
dhi.io/postgresql

0H

AWS S3
aws-sdk v3

✓

Kafka
dhi.io/kafka

0H

Inventory service

Other services

Total stack exposure

0C · 0H CVEs

Three problems every security team knows



Integrity

How do you know every component is exactly what it claims to be and has not been tampered with in transit?



Excessive Attack Surface

General-purpose base images ship 500+ packages, most unused by your app. Every package is a potential CVE.



Operational Overhead

Security teams flooded with CVEs. Developers spending more time patching than building. Real work grinds to a halt.

Software Supply Chain risk in the AI age

infoq.com

NPM Ecosystem Suffers Two AI-Enabled Credential Stealing Supply Chain Attacks

The Node Package Manager (npm) ecosystem has suffered from two major supply chain attacks in recent months, affecting hundreds of packages...

Ars Technica

AI-generated code could be a disaster for the software supply chain. Here's why.

AI-generated computer code is rife with references to non-existent third-party libraries, creating a golden opportunity for supply-chain attacks.

29 Apr 2025

The Register

AI code suggestions sabotage software supply chain

The rise of LLM-powered code generation tools is reshaping how developers write software - and introducing new risks to the software supply chain in the...

BleepingComputer

AI-hallucinated code dependencies become new supply chain risk

A new class of supply chain attacks named 'slopsquatting' has emerged from the increased use of generative AI tools for coding and the model's tendency to...

IBM

How cyber criminals are compromising AI software supply chains

With the adoption of AI soaring across industries and use cases, preventing AI-driven software supply chain attacks has never been more important.

3 Apr 2025

www.trendmicro.com

Exploiting Trust in Open-Source AI: The Hidden Supply Chain Risk No One Is Watching

As open-source AI models become foundational to digital infrastructure, hidden backdoors and tampered supply chains pose a growing...

25 Jul 2025



The security gap is already here

State of Agentic AI Report ·
~~2026~~ developers & tech leaders

60%

already have AI agents in
production

94% call it a strategic priority

40%

cite security as the #1
challenge in scaling
agentic AI

45% can't ensure tools are
secure

85%

know MCP but most can't
deploy it securely at scale

Security & config issues block
production

94%

use containers for agent
development or
production

98% follow cloud-native
workflows

Software Supply Chain Security Challenges

96%

of applications include open source components

70–90%

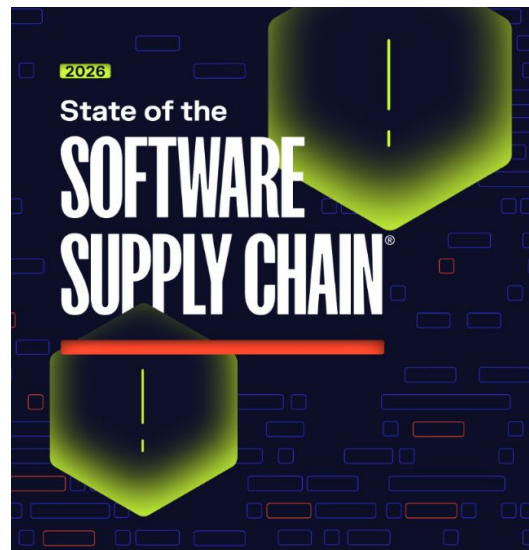
of code in an application is from open source

454,648

new malicious packages identified in 2025 alone

1,23,219

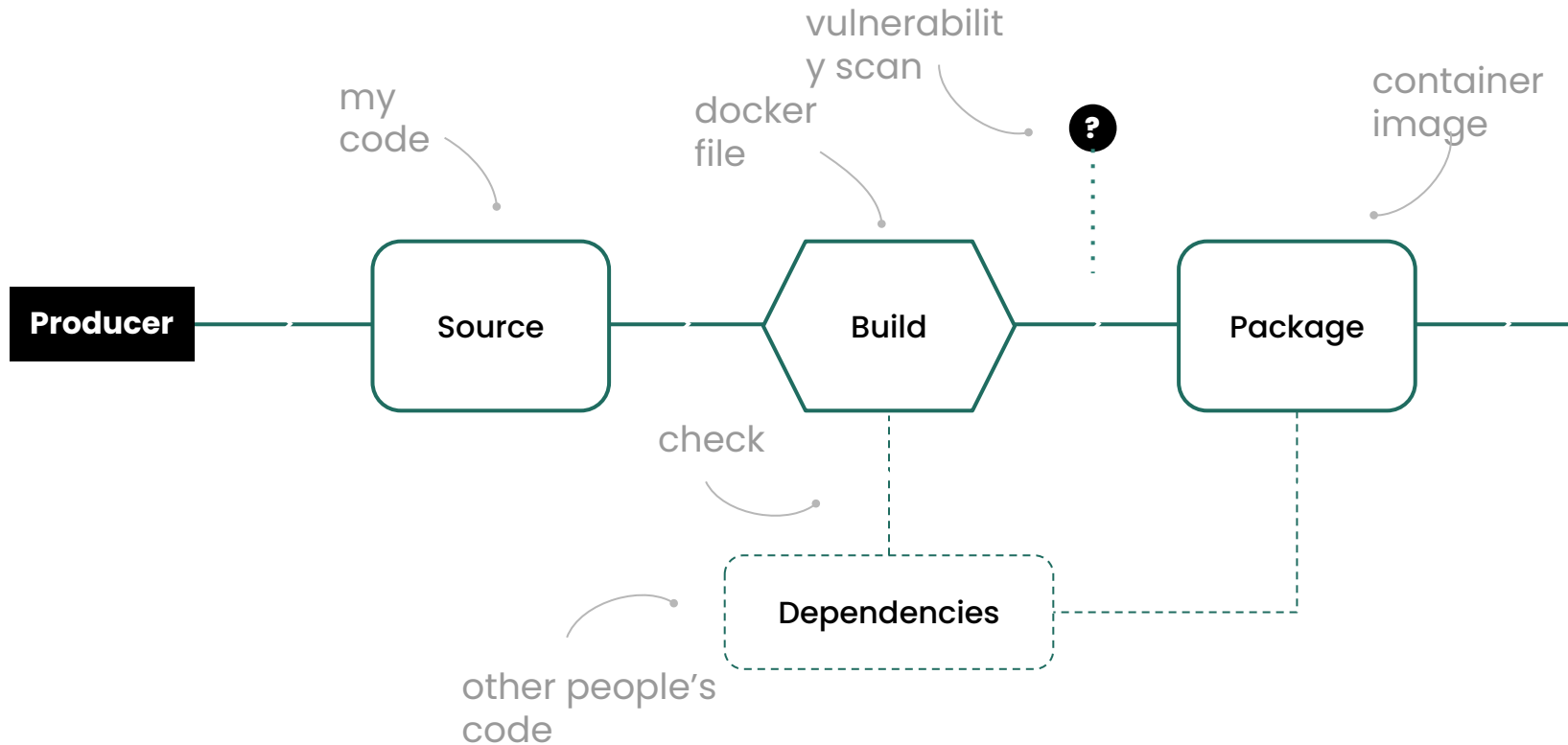
open-source malware packages logged since 2019

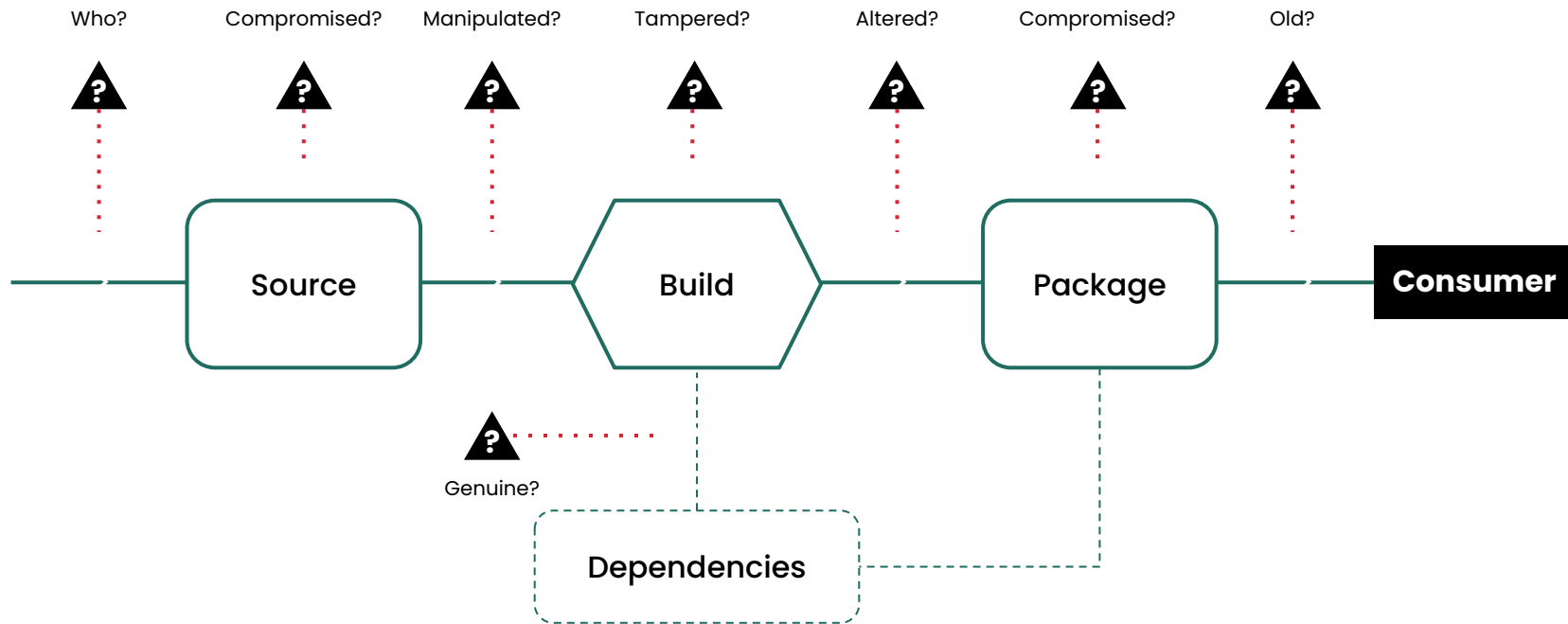


Source ~ [The State of Software Supply Chain Report 2026 by Sonatype](#)



Where things can go wrong in Supply Chain?





What's in this software artifact?

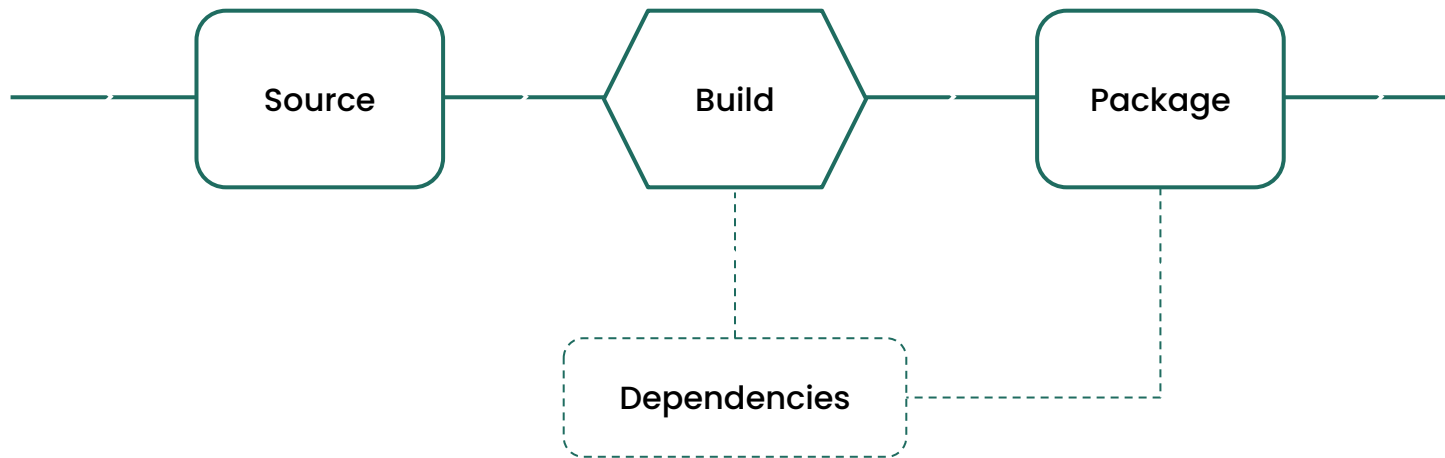
A software bill of materials – **SBOM** – is an *attestation* describing the contents of a software artifact

Where has it come from?

Provenance is an *attestation* about the history of an artifact – where it came from, who produced it, and how

Can I verify the attestation source?

An *attestation* that has been **signed** with a digital signature verifies the source and helps assess trustworthiness





Building Blocks

SBOM. VEX. SLSA

Especially when agents are doing the pulling.

Building Blocks of Vulnerability Management

SLSA

Where did it come from, and can you prove it?

Proof of where an artifact came from and that nobody tampered with it on the way.

SBOM

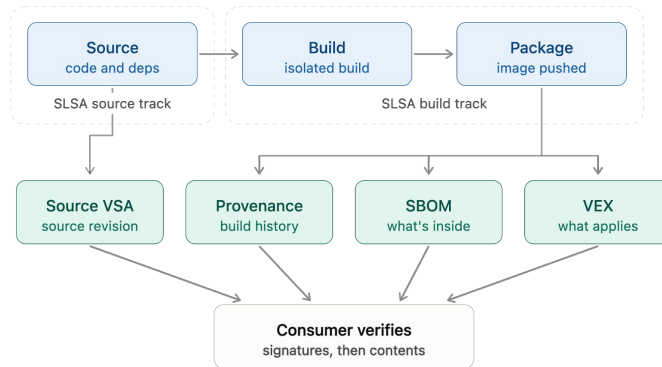
What's inside this thing?

The ingredient list: every package and version inside the image.

VEX

Which of these problems is actually my problem?

The supplier telling you which of those CVEs actually affect you.



5 categories ~ all built into DHI

• SECURITY METADATA & ATTESTATIONS

SBOM Complete software ingredient list (CycloneDX + SPDX)

SLSA provenance Tamper-proof build record, Level 3

Image provenance Source, commit, build environment

• VULNERABILITY & RISK MANAGEMENT

CVEs Near-zero critical/high, 7-day SLA

VEX Publisher declares non-exploitable CVEs

SSCS / SSDLC End-to-end supply chain + SSDLC

VERIFICATION AND TRACEABILITY

Code signing Every DHI signed with Cosign (Sigstore)

Image digests Pin exact image, guarantee no drift

Immutability Read-only filesystem, tamper-resistant

IMAGE STRUCTURE

Distroless
No shell, no package manager

Hardening
Minimal, non-root, secure default

glibc / musl
Debian + Alpine variants

COMPLIANCE

FIPS 140
Validated crypto modules

STIG
DoD hardening attestations

CIS Benchmark
Secure config baseline

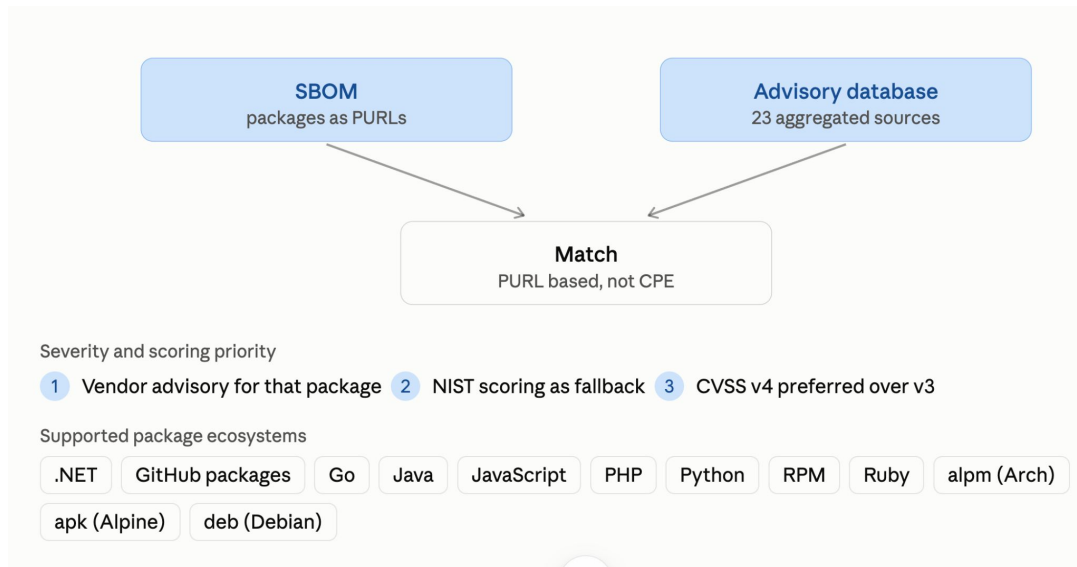




SBOM

Software Bill of Materials – your software ingredient list

- The SBOM contain a list of all packages in the image
- Image analysis uses image SBOMs to understand what packages and versions an image contains.
- Docker Scout uses SBOM attestations if available on the image. If no SBOM attestations is available, Scout creates one by indexing the image contents.



<https://docs.docker.com/scout/deep-dive/advisory-db-sources/>



SBOM

Software Bill of Materials – your software ingredient list

- The SBOM contain a list of all packages in the image
- Image analysis uses image SBOMs to understand what packages and versions an image contains.
- Docker Scout uses SBOM attestations if available on the image. If no SBOM attestations is available, Scout creates one by indexing the image contents.

```
# Generate an SBOM during the build time
```

```
$ docker buildx build \  
  --attest \  
  --type=sbom .
```

```
# By default, Buildkit uses the Syft-based  
scanner plugin to generate the SBOM. You can  
specify a different generator plugin with the  
generator if needed
```

```
$ docker buildx build \  
  --attest \  
  --type=sbom,generator=<image> .
```

```
# Query via Docker Scout
```

```
$ docker scout sbom myorg/myapp:v1.0
```

```
# Write SBOM to a file
```

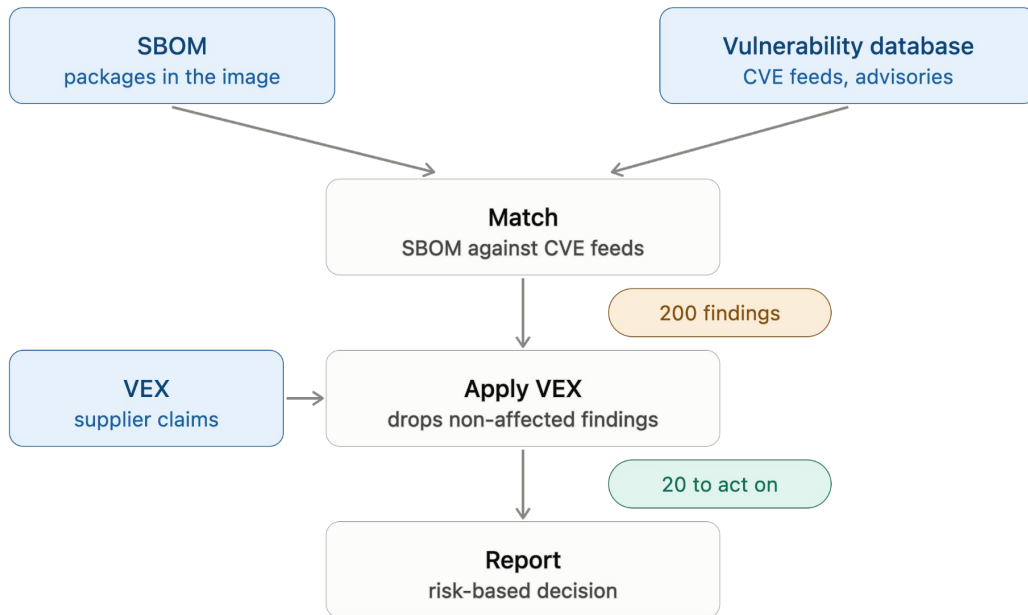
```
$ docker scout sbom -output alpine.sbom alpine
```



VEX

Vulnerability Exploitability eXchange, cutting through CVE noise

- A form of security advisory that indicates whether a products are affected by a known vulnerabilities.
- Instead of just listing all CVEs found in the image, VEX statements clarify whether each vulnerability is actually exploitable in the specific product configuration, and why.



VEX

Vulnerability Exploitability eXchange, cutting through CVE noise

WITHOUT VEX

- A standard vulnerability scan lists all CVEs found in the image, regardless of whether they are exploitable or not.

NAME	INSTALLED	FIXED IN	TYPE	VULNERAB
libc6	2.41-12+deb13u2		deb	CVE-2018
libc6	2.41-12+deb13u2	(won't fix)	deb	CVE-2026
libc6	2.41-12+deb13u2	(won't fix)	deb	CVE-2026
...				

The output shows all CVEs reported against packages present in the image, but doesn't indicate whether if they are really exploitable.

WITH VEX

A VEX-enabled scan provides statements for each CVE, such as:

- Not Affected: vulnerabilities are present but not exploitable in this image
- Affected: vulnerability is exploitable and action is recommended.
- Fixed: vulnerability has been fixed in this version
- Under investigation: The status is being determined

```
$ docker scout vex get dhi.io/python:3.13 --output vex.json
```

"190 not affected. 10 fixed."



SLSA

Supply chain Levels for Software Artifacts – build provenance

- A security framework designed to improve the integrity and security of software supply chains.
- Defines four progressive build levels (0–3), each adding more rigorous security controls around build provenance, source integrity, and build environment security.
- It helps prevent tampering, ensures artifact integrity, and secures build environments.
- SLSA is important for mitigating supply chain risks and ensuring transparency and accountability in software builds.

L0 No guarantees

L1 Provenance exists – build metadata documented

L2 Hosted build + signed provenance
GitHub Actions with OIDC achieves this

L3 Hardened build, non-falsifiable
provenance DHI target level

Can you prove this artifact came from that source, and was not tampered with in transit? SLSA makes that question answerable.

DHI GIVES YOU

- ✓ Signed provenance envelope
- ✓ Hermetic, reproducible builds
- ✓ Verifiable with Cosign / Notation



SLSA

Supply chain Levels for Software Artifacts – build provenance

- A security framework designed to improve the integrity and security of software supply chains.
- Defines four progressive build levels (0–3), each adding more rigorous security controls around build provenance, source integrity, and build environment security.
- It helps prevent tampering, ensures artifact integrity, and secures build environments.
- SLSA is important for mitigating supply chain risks and ensuring transparency and accountability in software builds.

```
# Retrieve and verify SLSA provenance  
attestations for Docker Hardened Images
```

```
$ docker scout attest get dhi.io/<image>:<tag> \  
  --predicate-type https://slsa.dev/provenance/v0.2 \  
  --verify
```

```
# fetches the SLSA attestation and verifies  
its authenticity
```



FIPS 140

Federal Information Processing Standard - validated cryptography

FIPS 140 mandates that cryptographic modules meet NIST-validated standards. Required for US government contracts, healthcare (HIPAA), finance, and defense workloads.

- ✓ DHI ships a `-fips` tag variant per image
- ✓ OpenSSL FIPS Provider - CMVP validated
- ✓ Signed FIPS attestation - machine-readable for auditors

```
# Pull the FIPS variant
FROM dhi.io/node:24-debian13-fips

# Inspect the signed FIPS attestation
docker scout attest get \
  --predicate-type https://docker.com/dhi/fips/v0.1 \
  --verify \
  dhi.io/node:24-debian13-fips

# Output shows certification details:
# "standard": "FIPS 140-3"
# "certification": "CMVP #4985"
# "status": "active"
```



STIG

Security Technical Implementation Guide - DoD hardening standard

STIGs define secure configuration baselines published by DISA for DoD environments. DISA has not yet published a container-specific STIG - Docker creates custom profiles based on the **GPOS SRG + DoD Container Hardening Process Guide**.

- ✓ DHI ships signed STIG scan attestations
- ✓ Reduces false positives common in container STIG scans
- ✓ STIG variant requires a Docker subscription

```
# Inspect the signed STIG attestation
docker scout attest get \
  --predicate-type https://docker.com/dhi/stig/v0.1 \
  --verify \    dhi.io/node:24-debian13-fips

# View all attestations on a DHI image
docker scout attest list \
  dhi.io/node:24-debian13

# Attestation types you'll see:
# SBOM · OpenVEX · SLSA · FIPS · STIG
# Scout health · Secrets scan
```



Standard Image Vs Docker Hardened Images

Watch the CVE count change with one FROM line swap.

What makes a hardened image hardened



Minimal

Built from source. Only the packages your runtime actually needs. No shell, no curl, no extras.

95% smaller



Attested

Every image ships with SBOM, VEX document, SLSA provenance, and a digital signature. Verify in one command.

SBOM ✓ · VEX ✓ · SLSA L3 ✓ ·
Signed ✓



Patched

Continuously updated. Near-zero CVEs on day one and the Docker team keeps it that way as new vulnerabilities emerge.

~0 CVEs on day one

Docker Official Image vs Docker Hardened Image

DOCKER OFFICIAL IMAGE

`node:22-slim`

Critical 2

High 26

Medium 25

Low 122

Packages 806

Size 398 MB

DOCKER HARDENED IMAGE

`dhi.io/node:24-debian13`

Critical 0

High 0

Medium 0

Low 0

Packages 211 (-595)

Size 40 MB (-90%)

```
docker scout compare catalog-service:dhi --to catalog-service:baseline --ignore-unchanged
```


catalog-service-node: migrate to DHI

```
# catalog-service-node/Dockerfile (before)
```

```
FROM node:22-slim AS base
```

```
WORKDIR /usr/local/app
```

```
RUN useradd -m appuser
```

```
USER appuser
```

```
COPY package*.json .
```

```
FROM base AS final
```

```
ENV NODE_ENV=production
```

```
RUN npm ci --production
```

```
COPY ./src ./src
```

```
EXPOSE 3000
```

```
CMD ["node", "src/index.js"]
```

```
# catalog-service-node/Dockerfile (after - DHI)
```

```
FROM dhi.io/node:24-debian13-dev AS base
```

```
WORKDIR /usr/local/app
```

```
COPY package*.json .
```

```
RUN npm ci --production
```

```
FROM dhi.io/node:24-debian13 AS final
```

```
ENV NODE_ENV=production
```

```
COPY --from=base /usr/local/app/node_modules
```

```
./node_modules
```

```
COPY ./src ./src
```

```
EXPOSE 3000
```

```
CMD ["node", "src/index.js"]
```

🕒 Runtime stage is distroless - no shell, no npm. Dev stage handles builds. Same source, same tooling. Compose file unchanged.

SBOM generation & signature verification

Making attestations tangible – not just theoretical.

BUILD WITH ATTESTATIONS

```
docker buildx build \  
  --sbom=true \  
  --provenance=mode=max \  
  -t YOUR_ORG/catalog-service:v1.0 .
```

INSPECT AND VERIFY

```
docker scout attest list catalog-service:dhi docker scout  
attest get \  
  --predicate-type https://slsa.dev/provenance/v0.2 \  
  --verify catalog-service:dhi docker scout policy  
catalog-service:dhi --exit-code
```

PART 3 OF 4

Securing Your CI Pipeline

Build policies · Image signing · GitHub Actions



Docker Scout build policies, security as code

Define rules that **automatically fail the build** before anything insecure reaches your registry or production.

POLICIES TO ENFORCE TODAY

- ✖ No fixable critical or high CVEs
- ✖ Supply chain attestations present
- ✖ No unapproved base images
- ✖ Default non-root user

✓ 7/7 policies passed - image reaches registry

```
# CLI - run against built-in policies, fail on
violation

docker scout policy catalog-service:dhi
--exit-code

# Optional: policy-config.json to customise thresholds

{
  "policies":
  [
    {
      "name": "fixable-vulnerabilities",
      "config":
      {
        "severities": ["CRITICAL", "HIGH"]
      }
    },
    { "name": "no-stale-base-images", "enabled":
false }
  ]
}
```

Image signing with Cosign

- 1 Build image with attestations
 - 2 Sign with Cosign (keyless, via OIDC)
 - 3 Signature stored in Sigstore transparency log
- ✓ Verify at deploy - CI gate or admission controller

Keyless signing via OIDC - no private key to manage · Works with Docker Hub, ECR, ACR, GHCR and any OCI registry

```
# Sign keylessly (OIDC - no private key)
cosign sign YOUR_ORG/catalog-service:v1.0

# Verify before deploy
cosign verify \
  --certificate-identity-regexp=".*" \
  --certificate-oidc-issuer="https://accounts.google.com" \
  YOUR_ORG/catalog-service:v1.0

# View attestations via Docker Scout
docker scout attest list YOUR_ORG/catalog-service:v1.0 #
Verify a specific attestation
docker scout attest get --verify \
  YOUR_ORG/catalog-service:v1.0
```

The complete secure CI pipeline



1. CHECKOUT

```
actions/ checkout@v4
```

2. BUILD + ATTEST

```
docker build --sbom=true  
--provenance=mode=max -t $IMAGE .
```

3. POLICY GATE ← KEY STEP

```
command: policy exit-on: policy #  
push never runs if this fails
```

4. PUSH

```
docker push "$IMAGE"  
attestations bound  
to digest
```

7 built-in Scout policies – zero config required

```
docker scout policy IMAGE --exit-code
```

No fixable critical or high CVEs

Critical/high CVEs that have a fix available

No high-profile vulnerabilities

Log4Shell, XZ backdoor, CISA KEV catalog

No copyleft licenses

AGPL, GPL, LGPL, MPL and similar

No outdated base images

Base image must match latest digest

Supply chain attestations

SBOM + SLSA provenance must be present

Default non-root user

Image must run as non-root

No unapproved base images

Base image must match an allowlist

 Configurable via JSON. Custom policies in Rego (OPA). Runs fully local – no Scout service needed.

The complete CI workflow – ready to copy

```
name: Secure Build
on: [push]
env:
  IMAGE: myorg/catalog-service:$
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # 1. Checkout
      - uses: actions/checkout@v4
      # 2. Login to Docker Hub
      - uses: docker/login-action@v4
        with:
          username: $
          password: $
      # 3. Build on DHI base + attach SBOM & provenance
      - name: Build with attestations
        run: |
          docker build -t $IMAGE \
            --sbom=true \
            --provenance=mode=max .
      # 4. Policy gate - before push
      # 7 built-in policies, runs local.
      # Fails → Push never runs.
      - name: Docker Scout policy
        uses: docker/scout-action@v1
        with:
          command: policy
          image: $
          organization: $
      # 5. Push - only runs if gate passes
      - name: Push
        run: docker push $IMAGE
```


CI gate outcomes: DHI vs. standard base

What happens when DHI base is used

- ✓ No critical/high CVEs
- ✓ SBOM + provenance
- ✓ Non-root (DHI default)
- ✓ Up-to-date base → **Gate passes**
- **Image pushed**

What happens with a standard base

- ✗ CVEs found
- ✗ No SBOM
- ✗ Root user → **Gate fails**
- **Push never runs**

CI policy enforcement in action

Watch a build fail – then pass – with one Dockerfile change.

ROUND 1 – FROM NODE:22-SLIM → FAIL

```
$ docker scout policy catalog-service:baseline✗ No fixable critical or high CVEs

2 critical, 26 high found✗ ■ Supply chain attestations

no SBOM or provenance present✗ Default non-root user

image runs as root✓
No copyleft licenses ✓
No high-profile vulnerabilities
Policy: FAILED (3 of 7 not met)
Push step skipped - image blocked
```

ROUND 2 – FROM DHI.IO/NODE:24-DEBIAN13 → PASS

```
$ docker scout policy catalog-service:dh✓
No fixable critical or high CVEs✓
Supply chain attestations

SBOM + provenance present✓
Default non-root user✓
No copyleft licenses✓
No high-profile vulnerabilities✓
No outdated base images✓
No unapproved base images
Policy: PASSED (7 of 7)
Image pushed → registry
```

PART 4 OF 4

Securing the Agentic Stack

MCP servers · Tool isolation · Trusted foundations



The better the agent, the bigger the blast radius

Agents execute code, call APIs, access filesystems – autonomously. One compromised MCP server can:

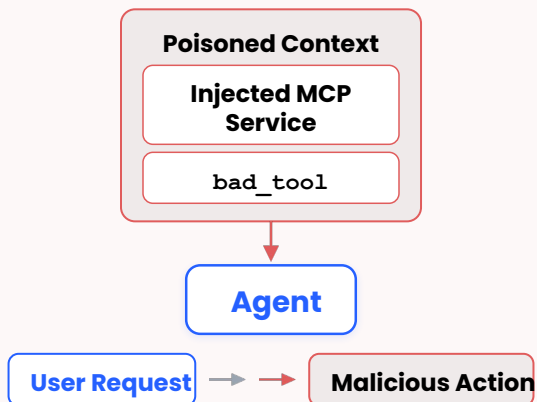
- ⚠ **Rug pull** server changes behavior after you approve it
- ⚠ **Tool poisoning** malicious prompts invisible to users, clear to AI
- ⚠ **Credential exfiltration** hardcoded env vars, easy target

CONTAINER ISOLATION = BLAST RADIUS BOUNDARY

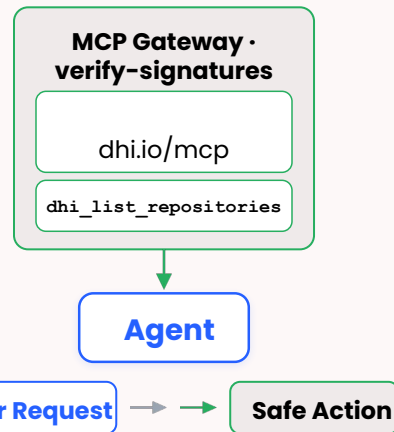
Run MCP servers in containers. Even a fully compromised MCP server cannot reach beyond its container boundary – no host filesystem, no sibling services, no credentials it was not explicitly given.

"Containers brought order to app deployment. They can do the same for agentic AI."

Without MCP Gateway – tool poisoning



Credentials exfiltrated · rug pull · prompt injection invisible to user



Unsigned server rejected by Gateway · only signed, attested tools run

MCP servers belong in hardened containers

1. BUILD ON DHI - SAME AS ANY APP

```
# mcp/Dockerfile FROM dhi.io/python:3.13 WORKDIR /app
COPY requirements.txt . RUN pip install --no-cache-dir
-r requirements.txt COPY src/ ./src/ # No shell. No
curl. No package manager. CMD ["python",
"src/catalog_mcp.py"]
```

```
services:  catalog-mcp:      build: .      read_only:
true      cap_drop: [ALL]    security_opt:
[no-new-privileges]
```

2. VERIFY & RUN THROUGH THE MCP GATEWAY

```
# Gateway verifies signature before running docker mcp
gateway run --verify-signatures # Check attestations on
your built image docker scout quickview catalog-mcp:dhi
docker scout attest list catalog-mcp:dhi
# Confirm runtime hardening applied docker inspect
catalog-mcp \
  --format
' '
```

Don't hand-harden every server

For off-the-shelf tools (filesystem, git, GitHub) use Docker's pre-hardened MCP Catalog - signed, attested, zero Dockerfile work:

```
docker mcp catalog pull  docker/mcp-catalog-dhi:latest
docker mcp catalog server ls  docker/mcp-catalog-dhi
```

Two paths – same agent, same prompt

TRADITIONAL AGENTIC SDLC

1 `claude -p "containerize catalog service"`



2 **Pulls packages from random internet repos**
npm registry · DockerHub community · apt · pip



3 `FROM node:20`
Dockerfile written – no security guidance



2C · 26H · 25M · 122L CVEs

806 packages · no SBOM · no attestation · root user

VS

AGENTIC SDLC WITH SBX + DHI MCP

1 `sbx run --name ws claude`
Agent runs inside an isolated microVM



2 **Queries DHI MCP Server first**
`dhi_list_repositories` · `dhi_get_image_cves`



3 `FROM dhi.io/node:24-debian13`
DHI-recommended base – 0 CVEs, signed, SLSA L3



0C · 0H · 0M · 0L CVEs

211 packages · SBOM attached · signed · non-root

DHI MCP Server – the agent chooses wisely

The agent that introduced vulnerabilities can now query what's secure – before picking a base image.

CONNECT YOUR AI ASSISTANT (ONE CONFIG)

```
# Claude Desktop - claude_desktop_config.json
{
  "mcpServers": {
    {
      "dhi": {
        "url": "https://dhi.io/mcp"
      }
    }
  }
}

# Claude Code
claude mcp add dhi --url https://dhi.io/mcp
```

WHAT YOU CAN ASK

"Find the Node.js hardened image with the fewest CVEs"

"Does dhi.io/node:24-debian13 have FIPS and STIG attestations?"

"What packages are in catalog-service:dhi and do any have known exploits?"

10 TOOLS AVAILABLE TO THE AGENT

[dhi_list_repositories](#)

Search catalog by name, FIPS, STIG

[dhi_get_image_cves](#)

CVEs with CVSS, EPSS, fix version

[dhi_get_image_packages](#)

Full SBOM – packages, licenses, purls

[dhi_get_image_attestations](#)

SBOM, provenance, FIPS, STIG

[dhi_get_repository](#)

Tags, platforms, vuln counts

[dhi_create_mirror](#)

Mirror DHI repo to your org

🔗 The agent that triggered vulnerabilities now has the tools to **never make that mistake again** – by querying DHI before every FROM line.

Docker Sandboxes – the agent's own blast radius boundary

The lab started with `claude -p` building the catalog service. Run it in a sandbox – and the agent cannot touch your host.

INSTALL AND RUN

```
# Install the sbx CLI
brew install docker/tap/sbx sbx login

# Run agent in an isolated microVM
cd ~/my-project sbx run --name my-sandbox claude

# Clone mode - host is read-only
sbx run --clone --name my-sandbox claude

# Review agent commits before merging
git fetch sandbox-my-sandbox git log
sandbox-my-sandbox/main
```

WHAT'S ISOLATED

- ✓ **Own Docker daemon**
Agent builds containers without host daemon access
- ✓ **Own filesystem + network**
MicroVM boundary – cannot escape to host
- ✓ **Network policies**
Open / Balanced / Locked Down – block outbound by default
- ✓ **Clone mode – host read-only**
Agent works on its own branch; you review and merge

docs.docker.com/ai/sandboxes · CLI: `sbx`

Wire the DHI MCP Server into the Sandbox

1. SET UP THE SANDBOX

```
# One-time host setup
brew install docker/tap/sbx
export SBX_MCP_URL=https://gateway.docker.com

# Start the sandbox daemon
sbx daemon stop && sbx daemon start -d

# Nothing wired in yet
sbx mcp --help
sbx mcp ls
```

```
# Register the DHI MCP server by URL
sbx mcp add remotedhi --url https://dhi.io/mcp
sbx mcp inspect remotedhi

Name:      remotedhi
Type:      remote
URL:       https://dhi.io/mcp
Transport: streamable-http
```

2. QUERY IT FROM THE AGENT

```
cd ~/workdemo
sbx run codex --static-mcp remotedhi

> /mcp
MCP Tools - mcp-gateway (Auth: Unsupported)
Tools: code-mode, mcp-exec,
remotedhi__dhi_get_image_cves,
remotedhi__dhi_get_image_details, + 8 more
```

10 tools now available to the agent

code-mode, mcp-exec, dhi_get_image_cves, dhi_get_image_details, + 6 more
read-only queries. Mutating tools (dhi_create_mirror, dhi_remove_mirror) are the ones
to scope with policy - next slide.

```
$ sbx mcp ls
remotedhi  remote  streamable-http
```

Govern the Tools First – Cedar Access Policy

1. QUICK UNBLOCK – LAB DEFAULT

```
# Permits every action on every resource -
# unblocks the lab, but it's governance turned off
permit(principal, action == MCP::Action::"register",
resource);
permit(principal, action == MCP::Action::"invokeTool",
resource);
permit(principal, action == MCP::Action::"invokePrimordial",
resource);
```

Why this matters:

This permits every principal against every resource for all three actions. Fine to unblock the lab - don't ship it as your exemplar. `invokePrimordial` (the gateway's built-ins) also gets a wide-open pass here, which production policy should never allow.

2. PRODUCTION-SCOPED (RECOMMENDED)

```
// Anyone in the org may register servers
permit(principal, action == MCP::Action::"register", resource);
// Read-only DHI tools are invocable - mirror
// mutation + primordials are not
permit(principal, action == MCP::Action::"invokeTool", resource)
when {
  resource.server == "remotedhi" &&
  ["dhi_get_image_cves", "dhi_get_image_details",
   "dhi_get_image_packages", "dhi_get_image attestations",
   "dhi_get_tag_definition", "dhi_get_repository",
   "dhi_list_repositories", "dhi_list_mirrors"].contains(resource.tool)
};
```

Scoped by design

`dhi_create_mirror`, `dhi_remove_mirror` and a wide-open `invokePrimordial` are left out - query the catalog, don't mutate it.

```
# The lab's own doc still says "invoke" -
# the real action name is invokeTool.
```

The complete agentic supply chain



Every step is verifiable. Every artifact is trusted.

MCP server on DHI – end to end

From Dockerfile to verified tool invocation.

BUILD

```
docker buildx build  
--sbom=true -t  
catalog-mcp:dhi .
```

SIGN

```
cosign sign catalog-mcp:dhi
```

RUN

```
docker compose up  
(read_only, cap_drop: ALL)
```

VERIFY

```
docker scout attest list  
catalog-mcp:dhi
```

Your security framework – try these steps

- 1 Know what is in your images** SBOM + VEX
- 2 Verify where they came from** SLSA provenance + image signing
- 3 Start from a trusted base** Docker Hardened Images
- 4 Enforce at the pipeline** Docker Scout build policies
- 5 Isolate your agents** MCP servers in hardened containers

Resources & next steps

**dock
er**

FROM THIS WORKSHOP

Lab repo

github.com/ajeetrainna/simspace-agentic-security

DOCKER DOCS

Docker Hardened Images

hub.docker.com - Trusted Content → Hardened Images

Docker Scout

docs.docker.com/scout

MCP Catalog

hub.docker.com/mcp

STANDARDS

SLSA framework

slsa.dev

VEX specification

openvex.dev

Cosign / Sigstore (image signing)

docs.sigstore.dev/cosign/overview

MCP specification

modelcontextprotocol.io



Find us at the Docker booth. Let us know what you build.

Thank You.

